

ЛАБОРАТОРНАЯ РАБОТА №5: «Аутентификация, Привилегии и Защита»

Легенда: Ты проводишь аудит безопасности свежестановленного сервера. Тебе нужно настроить безопасный удаленный доступ, проверить уязвимости в правах sudo и убедиться, что системы принудительного контроля работают.

Задача 1. Безопасная настройка SSH

Шаг 1.1. Сгенерируй пару SSH-ключей (тип ed25519) с комментарием "security_lab".

```
(kali@kali)-[~]
$ ssh-keygen -t ed25519 -C "security_lab"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kali/.ssh/id_ed25519): /home/kali/.ssh/id_ed25519
Enter passphrase for "/home/kali/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_ed25519
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:WCwA9SWGJqGOMPfi2KwJrqIaJyVJaqXR3mULLFKZZPI security_lab
The key's randomart image is:
+--[ED25519 256]--+
|  o==* ...      |
| .o.*B =o       |
|+o.=.E+oo       |
|*+= .. o+       |
|+=o.... S       |
|.B .            |
|= =             |
|+*              |
|@.              |
+---[SHA256]-----+
```

воспользовались командой `ssh-keygen` с флагом `-t`, который показывает тип шифрования (в нашем случае это `ed25519`) и флаг `-C` для комментария (по нашему заданию комментарий `security_lab` был успешно добавлен). ключи приватный и публичный сохранены в директориях `/home/kali/.ssh/id_ed25519` и `/home/kali/.ssh/id_ed25519.pub` соответственно.

Шаг 1.2. Скопируй публичный ключ в `authorized_keys` для своего пользователя (используй `ssh-copy-id` на `localhost`).

воспользовались командой `ssh-copy-id -i ~/.ssh/id_ed25519.pub kali@127.0.0.1`

```
(kali@kali)-[~]
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub kali@127.0.0.1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_ed25519.pub"
The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ED25519 key fingerprint is: SHA256:AWEE6fCbLM66M6yPyLj9gfcJ9D6vC/h4vIW6c9uzkEEA
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kali@127.0.0.1's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/kali/.ssh/id_ed25519 'kali@127.0.0.1'"
and check to make sure that only the key(s) you wanted were added.
```

Шаг 1.3. Отредактируй /etc/ssh/sshd_config:

открыли файл в текстовом редакторе nano при помощи команды `sudo nano /etc/ssh/sshd_config`:

```
(kali@kali)-[~]
$ sudo nano /etc/ssh/sshd_config
[sudo] password for kali:
```

далее внутри редактора nano и сочетания клавиш Ctrl+W искали среди многих строк нужные нам

- Запрети вход по паролям (PasswordAuthentication no).

ввели Ctrl+W ввели PasswordAuthentication, нашли его, раскомментировали и заменили yes на no

```
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to "no" h
PasswordAuthentication no
#PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authenticat
```

- Запрети прямой вход под root (PermitRootLogin no).

далее все аналогично с пунктом предыдущим

```
# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
```

- (Опционально) Смени порт с 22 на 2222.

```
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

далее при помощи комбинации клавиш Ctrl+O сохранили изменения, затем при помощи Ctrl+X закрыли редактор

Шаг 1.4. Перезапусти службу SSH и проверь, что ты можешь зайти по ключу, но не можешь по паролю.

```
(kali@kali)-[~]
$ ssh kali@127.0.0.1
Linux kali 6.18.12+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 15 13:24:32 2026 from 192.168.0.105
```

попробовали войти и успешно! пароль не запросило, публичный ключ запоминать или куда-то переписывать не надо, так как мы их сохранили в папке по умолчанию /home/kali/.ssh/id_ed25519, поэтому ssh служба сама нашла наш приватный ключ.

Задача 2. Аудит и эксплуатация SUDO

Шаг 2.1. Создай нового пользователя testuser с паролем.

```
└─(kali㉿kali)-[~]
└─$ sudo adduser testuser
[sudo] password for kali:
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for testuser
Enter the new value, or press ENTER for the default
    Full Name []: Maria
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
```

создали нового пользователя testuser с паролем. воспользовались командой `sudo adduser testuser`, далее задали пароль, полное имя, а остальное пропустили.

Шаг 2.2. Используя visudo, дай testuser право выполнять команду `/usr/bin/vim` от имени root **без пароля** (NOPASSWD).

вот у меня получилась такая команда:

testuser ALL = (root) NOPASSWD: /usr/bin/vim

```
└─(kali㉿kali)-[~]
└─$ sudo visudo
```

```
# User privilege specification
root    ALL=(ALL:ALL) ALL

testuser ALL = (root) NOPASSWD: /usr/bin/vim

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL
```

далее сохранили и закрыли редактор как обычно.

Шаг 2.3. Переключись на testuser (`su - testuser`).

```
(kali@kali)-[~]
$ su - testuser
Password:
(testuser@kali)-[~]
$
```

заменяли юзера.

Шаг 2.4. Продемонстрируй повышение привилегий: используя разрешенный vim, получи оболочку root (подсказка: используй GTF0Bins для vim).

```
(testuser@kali)-[~]
$ sudo /usr/bin/vim -c ':sh'

(root@kali)-[/home/testuser]
# whoami
root
```

воспользовались командой `sudo /usr/bin/vim -c ':sh'`, запустилась оболочка vim, далее мы командой `whoami` проверили, кто мы, а мы root! получается повысили привилегии)

Шаг 2.5. Проверь свои текущие права sudo командой `sudo -l`.

проверяем свои текущие права:

команда `sudo -l`

```
(root@kali)-[/home/testuser]
# sudo -l
Matching Defaults entries for root on kali:
  env_reset, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User root may run the following commands on kali:
  (ALL : ALL) ALL
```

проверили, что мы как root можем выполнять абсолютно все программы.

Задача 3. Знакомство с PAM и 2FA (Демонстрация)

Шаг 3.1. Установи модуль Google Authenticator (`sudo apt install libpam-google-authenticator`).


```

(kali@kali)-[~]
$ sudo apt install libpam-google-authenticator
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  libdnnl3.6 libcud76 libonnxruntime-providers libonnxruntime1.21 libxnnpack0.20241108 libzxing3 python3-pysnmp4
Use 'sudo apt autoremove' to remove them.

Installing:
  libpam-google-authenticator

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1406
  Download size: 46.9 kB
  Space needed: 132 kB / 60.8 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 libpam-google-authenticator amd64 20250213-1.11-0.1 [46.9 kB]
Fetched 46.9 kB in 1s (68.9 kB/s)
Selecting previously unselected package libpam-google-authenticator.
(Reading database... 426275 files and directories currently installed.)
Preparing to unpack .../libpam-google-authenticator_20250213-1.11-0.1_amd64.deb...
Unpacking libpam-google-authenticator (20250213-1.11-0.1)...
Setting up libpam-google-authenticator (20250213-1.11-0.1)...
Processing triggers for kali-menu (2026.2.2)...
Processing triggers for man-db (2.13.1-1)...

```

установка прошла успешно

Шаг 3.2. Запусти настройку для своего пользователя (google-authenticator). Ответь на вопросы (сохрани резервные коды!).

```

(kali@kali)-[~]
$ su - testuser
Password:
(testuser@kali)-[~]
$ google-authenticator
Do you want authentication tokens to be time-based (y/n) y

```



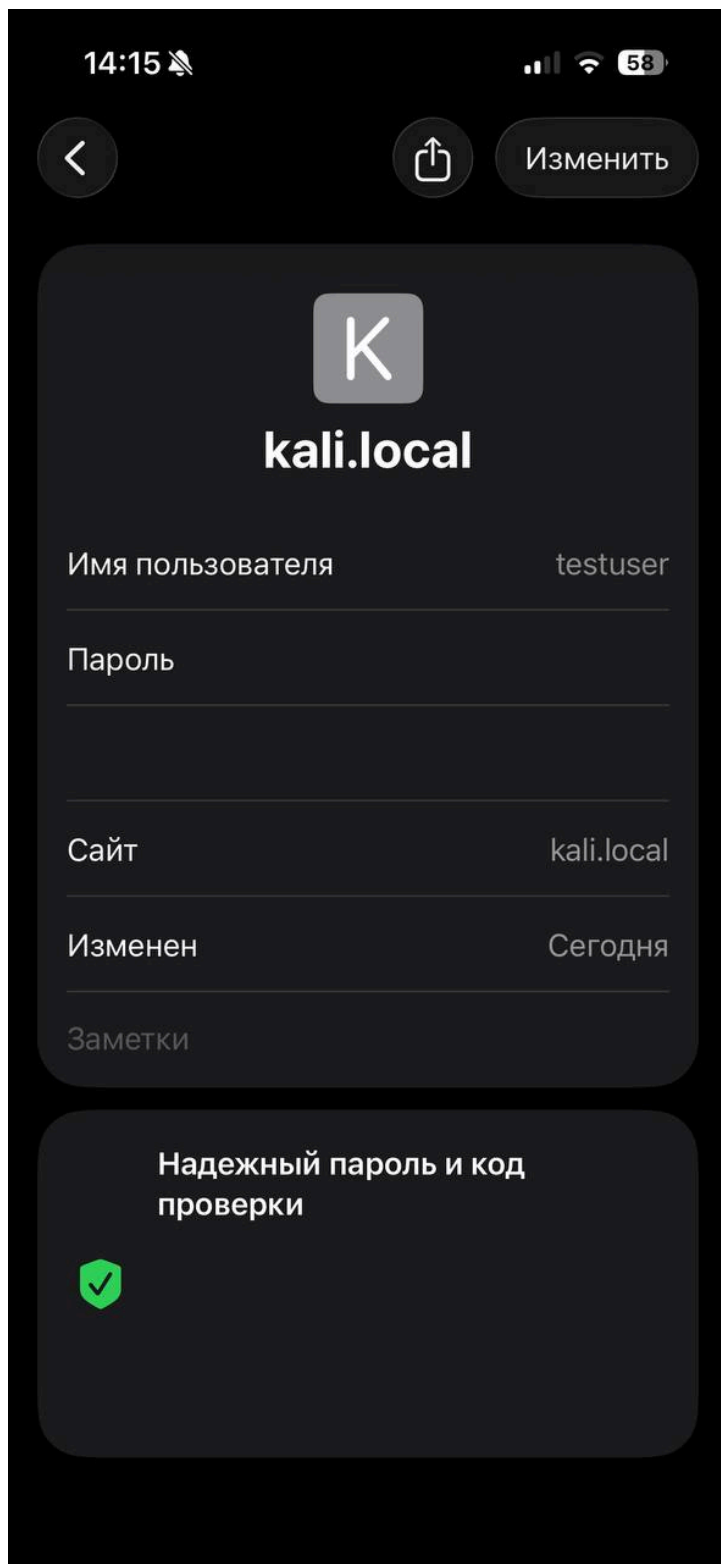
```

Your new secret key is: WCV
Enter code from app (-1 to skip):

```

вот мне выдали секретный ключ

далее на телефоне отсканировала qr код создала пользователя и создала 6-значный код, который меняется каждые 30 секунд



далее получила ключ секретный и свои резервные коды, но их я не покажу)))

```
Do you want me to update your "/home/testuser/.google_authenticator" file? (y/n)

Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
Do you want to do so? (y/n) y

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y
```

везде поотвечала у, мне дали еще резервные коды.

Шаг 3.3. Добавь строку `auth required pam_google_authenticator.so` в начало файла `/etc/pam.d/sudo`.

```
(testuser@kali) - [~]
$ sudo /usr/bin/vim /etc/pam.d/sudo
```

```
##PAM-1.0
# Home

# Set up user limits from /etc/security/limits.conf.
session    required    pam_limits.so
auth required pam_google_authenticator.so
@include common-auth
@include common-account
@include common-session-noninteractive
```

ввели строчку в начало, все как надо. чтобы выйти из режима редактирования и все сохранить зажали Shift и ввели ZZ.

Шаг 3.4. Попробуй выполнить любую команду через `sudo`. Убедись, что система теперь требует не только пароль, но и 6-значный код (или резервный код).


```
(testuser@kali)-[~]  
$ sudo whoami  
Verification code:  
[sudo] password for testuser:  
Sorry, user testuser is not allowed to execute '/usr/bin/whoami' as root on kali.
```

запросило код, но мы слишком сильно ограничили права testuser, поэтому нам выдало такое сообщение.

Шаг 3.5. Важно: После проверки удали эту строку из /etc/pam.d/sudo, чтобы не сломать себе доступ к sudo навсегда!

удалила строчку, заново открыла vim

Задача 4. Анализ AppArmor

Шаг 4.1. Проверь статус AppArmor в системе (sudo aa-status). Сколько профилей загружено? Сколько процессов находится под защитой?

сначала надо вернуть права для testuser на все

сначала я стала суперпользователем

```
# visudo  
#
```

```
root    ALL=(ALL:ALL) ALL  
  
testuser ALL=(ALL:ALL) ALL  
  
# Allow members of group sudo to execute any command  
%sudo   ALL=(ALL:ALL) ALL
```

далее ввели команду sudo aa-status

```
(testuser@kali)-[~]  
$ sudo aa-status  
[sudo] password for testuser:  
apparmor module is loaded.  
Failed to get profiles: 2....
```

получили, что у нас два профиля, так оно и есть: kali и testuser

Шаг 4.2. Посмотри логи заблокированных действий за последние 10 минут (`sudo dmesg -T | grep DENIED` или `journalctl -k | grep DENIED`).

```
(testuser@kali)-[~]  
$ sudo dmesg -T | grep DENIED  
  
(testuser@kali)-[~]
```

блокировок нет, в системном логе ядра сейчас просто отсутствуют записи, содержащие слово DENIED. система работает штатно, AppArmor ничего не запрещал.